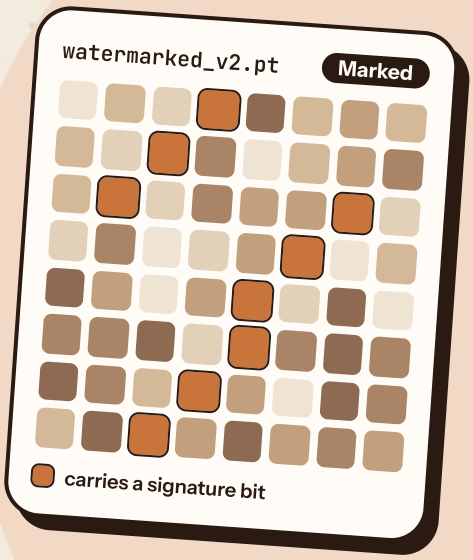
sigil



OWASP AI MODEL WATERMARKING
FINAL-YEAR PROJECT 2026–27

User guide & product explained.

How to install, run and present the Sigil demo, what every screen means, and how the hidden watermark and the zero-knowledge proof of ownership work underneath.

01 Embed → 02 Commit → 03 Prove → 04 Verify

Sami Daire
231813
Project lead · ZK & verification

Mohamed Ansari
231803
AI & watermarking

Arfat Shaikh
231846
Frontend & integration

Yash Bandiwadekar
231810
Backend developer

Guided by Er. Fiza Shaikh (Co-Guide) and the Department Review Committee.

Demo build · October 2026. A student project aligned with the OWASP AI Model Watermarking incubator project; not official OWASP work.

CONTENTS

What's in this guide

Start with chapter 1 for the idea, chapter 5 to get it running, and chapter 7 for step-by-step walkthroughs. Chapters 8–11 are reference.

1 **Sigil in one page**
The problem, the idea, the headline numbers

2 **Key ideas in plain words**
Every term the app uses

3 **How it works**
Embed, commit, prove, verify; why a thief can't win

4 **What's in the package**
The folders in the zip

5 **Installing and starting**
Quick start in 1 minute, or everything live

6 **A tour of the app**
Every page and control, with screenshots

7 **Step-by-step walkthroughs**
Presenting, verifying, attacking, showing it live

8 **The SDK and command line**
Watermark and verify your own models

9 **Results at a glance**
All measured numbers in one place

10 **Pass and fail cases**
What gets accepted, what gets rejected, and why

11 **Honest limits**
What the demo does not claim

12 **Troubleshooting and FAQ**
Fixes for common problems

13 **Team and credits**
Who built what

Conventions

`Monospace` is something you type or a file name. **Bold** names a button or label on screen. Addresses like `/app/verify` are pages of the app at `http://127.0.0.1:8765`. Every number in this guide comes from the demo's saved results (`demo/results/`), the same ones the app shows.

CHAPTER 1

1 Sigil in one page

Sigil hides a cryptographic watermark inside a neural network's weights, then proves who owns the model with a zero-knowledge proof. Anyone can check the proof; nobody learns the watermark.

The problem

A trained model is expensive: data, compute and expertise. Once a copy leaks, or a customer passes it on, the owner needs a way to say *"that model is mine"* and have a third party (a court, a marketplace, an auditor) check it.

A **white-box watermark** does part of this: during training, the owner nudges one layer's weights so that a secret key reads out a secret bit pattern (the signature). Later the owner extracts the pattern from a suspect copy. But this has three weaknesses, and the demo shows each one happening:

- **Checking reveals the secret.** To verify, the checker normally needs the key and signature, and whoever has them can scrub the mark from the model.
- **Anyone can claim anything.** A thief can invent a key that "finds" their own message in the stolen model, or even in a model that was never watermarked (demo C: the thief reads out "MALLORY!" with zero errors).
- **A valid proof isn't enough.** A zero-knowledge proof only says "some computation was correct". Without binding it to the owner, the exact model and the moment of the claim, a forged or replayed proof also verifies (demo E).

What Sigil does

01 Embed
A secret seed generates the key and a 64-bit signature. Training hides the signature in one layer. Accuracy barely moves.

02 Commit
Before release, the owner publishes a Poseidon hash of the seed, signs it with their Ed25519 key and gets a registry timestamp.

03 Prove
A Circom circuit re-derives the key and signature from the committed seed and checks the suspect's weights. SnarkJS turns that into a Groth16 proof.

04 Verify
Anyone checks the proof in milliseconds, off-chain or on an Ethereum-style chain. Zero watermark bits are revealed.

Headline numbers

ACCURACY COST

-0.03 pp

98.69% clean → 98.66% watermarked

PROOF

3.73 s

to prove; 879-byte proof; verifies off-chain and on-chain (243,225 gas)

REVEALED TO THE VERIFIER

0 of 64

watermark bits

FORGERIES ACCEPTED

0 / 20,000

thief seed search against the committed seed

ATTACKS SURVIVED

13 / 14

pruning, quantisation, fine-tuning, overwriting; feature permutation is the known exception

MISUSE REJECTED

4 / 4

replay, wrong model, tampered proof, untrusted registry

Who it's for

- **Model owners** who want to protect a model before releasing it.
- **Verifiers** (courts, marketplaces, auditors) who need to check an ownership claim without learning the owner's secrets.
- **Examiners and researchers**: the six research demos (A–F) walk through the attacks on naive schemes and how each fix closes them.

CHAPTER 2

2 Key ideas in plain words

The app uses these terms everywhere. Each is explained without maths first; chapter 3 adds the detail.

TERM	WHAT IT MEANS HERE
Weights	The numbers a neural network learns during training. The demo model is a small convolutional network for MNIST handwritten digits.
Watermarked layer	The layer that carries the mark: `conv2`. Its weights are averaged into a list of **288 numbers** (the "Uchida vector", after the method's inventor).
Signature	The secret bit pattern hidden in the layer: **64 bits** (T = 64) in the demo; 256 is the security setting.
Key X	A secret 64 × 288 table of +1/−1 values. Multiplying it by the layer's 288 numbers gives 64 results; each result's sign is one extracted bit.
Secret seed	A 248-bit random number that generates both the key and the signature. It never leaves the owner's machine (stored encrypted).
BER (bit error rate)	The fraction of the 64 bits that read out wrong. 0.000 means a perfect match; about 0.5 means random (no watermark).
Threshold θ	The acceptance rule: at most **8 of 64** bits may be wrong (θ = 0.125). In protocol v2 the circuit enforces it.
Commitment	A hash of the seed (plus the owner's key and the settings) published before release. It reveals nothing about the seed, and the seed can't be swapped for another one later.
Poseidon	A hash function designed to be cheap inside zero-knowledge circuits. Sigil's Python version matches circomlib bit-for-bit.
Owner identity	An Ed25519 key pair (elliptic-curve signatures). The owner signs their registration with it, and the seed's key and signature depend on it.
Registry receipt	A timestamp signed by a registry: "this registration existed at this time". When two parties claim the same model, the earliest valid registration wins.
Claim and nonce	The verifier opens a claim about a suspect file with a fresh random number (the nonce). The proof must include it, so an old proof can't be replayed.
w_hash	A hash of the suspect model's layer values, computed inside the proof *and* recomputed by the verifier from its own copy of the file. It ties the proof to that exact model.
zk-SNARK, Groth16	A short proof that a computation was done correctly on secret inputs, revealing nothing else. Groth16 is the proof system used: 3 elliptic-curve points (879 bytes as JSON).
Circom, SnarkJS	Circom describes the computation as a circuit (`circuits/ownership.circom`); SnarkJS creates and checks the proofs.

TERM	WHAT IT MEANS HERE
Trusted setup	Groth16 needs public parameters made once per circuit. Phase 1 uses the public Perpetual Powers of Tau ceremony; phase 2 is one contribution for Sigil's circuit.
On-chain verifier	A Solidity smart contract exported by SnarkJS that checks the same proof on an Ethereum-style chain. The demo runs it on a local Hardhat chain.
Protocol v1 / v2	v1 is the research baseline (demos A–E). v2 is the full fix (demo F and the SDK). See §3.7.
Replay (simulate) mode	The app shows the saved results and the Run buttons replay recorded logs; nothing executes. Live mode runs the real scripts on your machine.

CHAPTER 3

3 How it works

Four parties, four steps, and eleven checks. The full specification is `docs/protocol.md`.

3.1 Who is involved

PARTY	HOLDS	TRUSTED FOR
Owner	The secret seed and an Ed25519 private key	Nothing: the protocol assumes the prover may be malicious (for example, claiming someone else's model)
Registry	An Ed25519 registry key	Issuing honest timestamps (receipts)
Verifier	The registry's public keys and its own copy of the suspect model	Nothing beyond running the checks
Thief	The stolen weights; may register their own seed and key, fine-tune, prune or overwrite	—

3.2 Step 1 · Embed the watermark

1. The owner creates an **Ed25519 key pair** (their identity) and a random **seed**.
2. From (seed, owner key), a Poseidon-based generator derives the **key X** (64 × 288 values of ±1) and the **signature b** (64 bits). Because both depend on the owner's key, a thief can't reuse someone else's seed with their own identity.
3. Training adds a small extra loss so that, for the `conv2` layer's 288-value vector w, the 64 signs of $X \cdot w$ spell out b. In the demo this costs 0.03 percentage points of accuracy (98.69% → 98.66%; 98.57% for the v2 model).

3.3 Step 2 · Commit and register (before release)

1. The owner computes the **commitment** C = Poseidon(seed, owner key hash, settings hash). The settings (64 bits, layer `conv2`, at most 8 errors, fixed-point scale) are inside it, so they can't be loosened later.
2. The owner signs a **registration**: public key, settings, commitment, the model file's SHA-256 and the time.
3. A **registry** checks the registration and signs a **receipt** with the time it saw it. The seed stays on the owner's machine, encrypted (scrypt + AES-256-GCM).

3.4 Step 3 · Prove ownership of a suspect model

1. The verifier opens a **claim** about a suspect file: it records the file's SHA-256 and a fresh random **nonce**.

2. The owner runs the circuit. **Secret inputs:** the seed and the suspect's layer values (as fixed-point integers). **Public inputs:** owner key hash, settings hash, commitment, w_hash, nonce.

3. Inside the circuit: re-derive X and b from the seed; recompute the commitment and check it equals the public one; hash the layer values into w_hash; extract the 64 bits; count mismatches with b; **enforce at most 8**; bind the nonce.

4. SnarkJS outputs a **Groth16 proof**: 879 bytes. The circuit has 116,822 constraints; on the demo laptop the witness takes 1.43 s and the proof 3.73 s.

3.5 Step 4 · Verify

The verifier accepts only if **every** check passes. Each failure reports its own reason, which is what the app's **What gets checked** list shows.

CHECK (AS SHOWN IN THE APP)	WHAT IT RULES OUT
Registration signed by the owner	A registration made up or edited by someone else
Registry receipt from a trusted registry	A fake or self-made timestamp
Registered before the claim	Registering a model only after the dispute started
Claim refers to this registration	Mixing a proof with another owner's registration
Claim refers to this suspect file	Proving something about a different file than the one in dispute
Proof is for the owner's key	Using someone else's identity
Same watermark configuration	Proving with looser settings (for example, more allowed errors)
Opens the published commitment	Using a seed other than the one committed before release
About this exact model (w_hash)	Reusing a proof about a different model; the verifier recomputes w_hash from its own copy
Made for this claim (nonce)	Replaying an old proof
Groth16 proof verifies	Tampered or invalid proof bytes

3.6 Why a thief can't win

- **They can't open the owner's commitment** without the seed, so they can't prove the owner's registration.
- **Registering their own seed afterwards doesn't help:** their own key reads 28 of 64 bits wrong on the stolen model (limit 8), so no proof is possible, and their registration would be later anyway.
- **Searching for a lucky seed is hopeless at scale:** each try succeeds with probability about 2.8×10^{-10}. Demo D tried 20,000 seeds: 0 accepted, best 16 of 64 wrong. At T = 256 the odds become about 2^{-120}.
- **Overwriting** (embedding their own mark on top) can give both sides a valid proof. The registry timestamp settles it: the earliest valid registration wins (demo F6).

3.7 Protocol v1 vs v2

	V1 · RESEARCH BASELINE (DEMOS A–E)	V2 · THE FULL FIX (DEMO F, SDK)
Commitment	SHA-256 of seed + owner name	Poseidon of seed + Ed25519 key hash + settings
Key and signature	Gaussian key from the seed (numpy); shown on screen for teaching	±1 key and signature from a Poseidon generator; never shown
Owner identity	A text label (`owner:yash`)	Ed25519 signature on the registration
Zero-knowledge proof	EZKL (Halo2/KZG) experiments, not bound to model or claim	Circom + Groth16; threshold, model (w_hash) and claim (nonce) bound in-circuit
Verification	Off-chain	Off-chain (SnarkJS) and on-chain (Solidity verifier)

CHAPTER 4

4 What's in the package

Unzipping `owasp-wm-demo.zip` gives one folder, `owasp-ai-model-watermarking`. These are the parts you'll use.

FILE OR FOLDER	WHAT IT IS
`Start-Demo-UI-Simulate.cmd`	Windows: double-click to open the app in replay mode (needs only Python)
`Start-Demo-UI.cmd`	Windows: double-click to open the app in live mode (after the setup)
`setup.ps1`, `setup.sh`	One-command setup for live runs (Windows / macOS and Linux)
`Sigil-User-Guide.pdf`	This guide
`DEMO.md`	The short presentation runbook
`demo/`	Demo scripts A–F, the local server (`server.py`), saved results (`demo/results/`) and the model checkpoints (`demo/ckpt/`)
`frontend/`	The web app's source (React + TypeScript); `frontend/dist/` is the ready-built app the server shows
`sdk/`	The `sigil` Python SDK and command line (chapter 8)
`circuits/`	The ownership circuit, its verification keys and setup records, and the on-chain verifier project
`dist/sigil-proving-keys-t64.zip`	Proving keys for demo F, installed and checked by the setup
`contract/`	The API contract between server and UI, and the recorded logs replay mode plays back
`docs/`	Protocol specification, threat model, reports, and this guide's source
`tests/`, `sdk/tests/`	Automated tests (server, protocol, circuit, adapters)
`REQUIREMENTS.md`, `WORKFLOW.md`	Project requirements and the build workflow

Demo secrets are public on purpose

The owner keys and seeds in `demo/ckpt/` are committed so the saved results can be reproduced. Treat them as demo values. Real secrets created with the CLI go in `secrets/`, which is never committed or zipped.

CHAPTER 5

5 Installing and starting

Two ways to run it. Option 1 is enough for viewing and presenting; Option 2 makes the Run buttons execute for real.

5.1 What you need

ITEM	OPTION 1 · REPLAY	OPTION 2 · EVERYTHING LIVE
Operating system	Windows 10/11 (double-click launchers), macOS or Linux (terminal commands)	
Python	3.10 or newer from python.org. On Windows, tick **"Add python.exe to PATH"** in the installer.	
Node.js	Not needed	20.19 or newer (nodejs.org), for live demo F (the circuit tools)
Internet	Not needed	During setup only (packages and tools download). The app itself never loads anything from the internet.
GPU	Optional. Everything runs on a CPU; add `--gpu` to the setup to use an NVIDIA GPU.	
Time	About 1 minute	About 5–15 minutes, mostly downloading

5.2 Option 1 · Quick start in replay mode

1. **Unzip** `owasp-wm-demo.zip` anywhere, for example into Downloads.
2. **Windows:** open the `owasp-ai-model-watermarking` folder and double-click `Start-Demo-UI-Simulate.cmd`.
   macOS / Linux: open a terminal in that folder and run `python3 demo/server.py --simulate --open`.
3. A console window opens and your browser opens `http://127.0.0.1:8765` on the home page. If the browser doesn't open, type that address yourself.
4. **Keep the console window open** while you use the app. Closing it (or pressing Ctrl+C in it) stops the app.

> **What replay mode means**
> Every page shows the saved results from real runs. The Run buttons replay the recorded output of those runs in the console, so presenting works on any laptop, with no installs and no risk. A banner at the top of the app says **Simulate mode** so nobody mistakes it for a live run.

5.3 Option 2 · Everything live

1. Install **Python 3.10+** and **Node.js 20.19+**.

2. Open a terminal in the `owasp-ai-model-watermarking` folder (Windows 11: right-click the folder → **Open in Terminal**).

3. Run the setup:

```
# Windows (PowerShell)
.\setup.ps1 --demo
# if PowerShell blocks the script:
powershell -ExecutionPolicy Bypass -File setup.ps1 --demo
# macOS / Linux
./setup.sh --demo
```

   It creates a private Python environment (`.venv`), installs a CPU build of PyTorch, the demo packages, the `sigil` SDK and the circuit tools, then installs the proving keys from `dist/`, checking each against the published records in `circuits/keys/`.

4. Start the app: double-click `Start-Demo-UI.cmd` (Windows) or run `.venv/bin/python demo/server.py --open` (macOS / Linux).

5. Check the sidebar's environment chip: it shows your GPU's name or **CPU only**. If it shows **No ML stack: replay only**, the setup didn't finish; run it again or use Option 1.

Setup options

OPTION	EFFECT
`--demo`	Everything the live demo needs (the usual choice)
`--gpu`	With `--demo`: install the CUDA build of PyTorch if an NVIDIA GPU is present
`--cpu`	Install the CPU build even if a GPU is present
`--no-tf`	Skip TensorFlow/Keras (saves about 1 GB; only the Keras adapter needs it)
`--minimal`	Install nothing: the app with saved results only
`--keys <zip>`	Install proving keys from a specific zip
`--zk-setup`	Generate the proving keys on this machine instead (about 3 minutes)

5.4 Stopping, ports and a backup

- **Stop:** close the console window, or press Ctrl+C in it.
- **Port in use:** start on another port, for example `python demo/server.py --port 8770`, and open `http://127.0.0.1:8770`.
- **Backup during a presentation:** keep a second terminal ready with `python demo/server.py --simulate --port 8766`. If a live run misbehaves, switch the browser to `http://127.0.0.1:8766`.
- The server only listens on `127.0.0.1`: other computers on the network can't reach it.

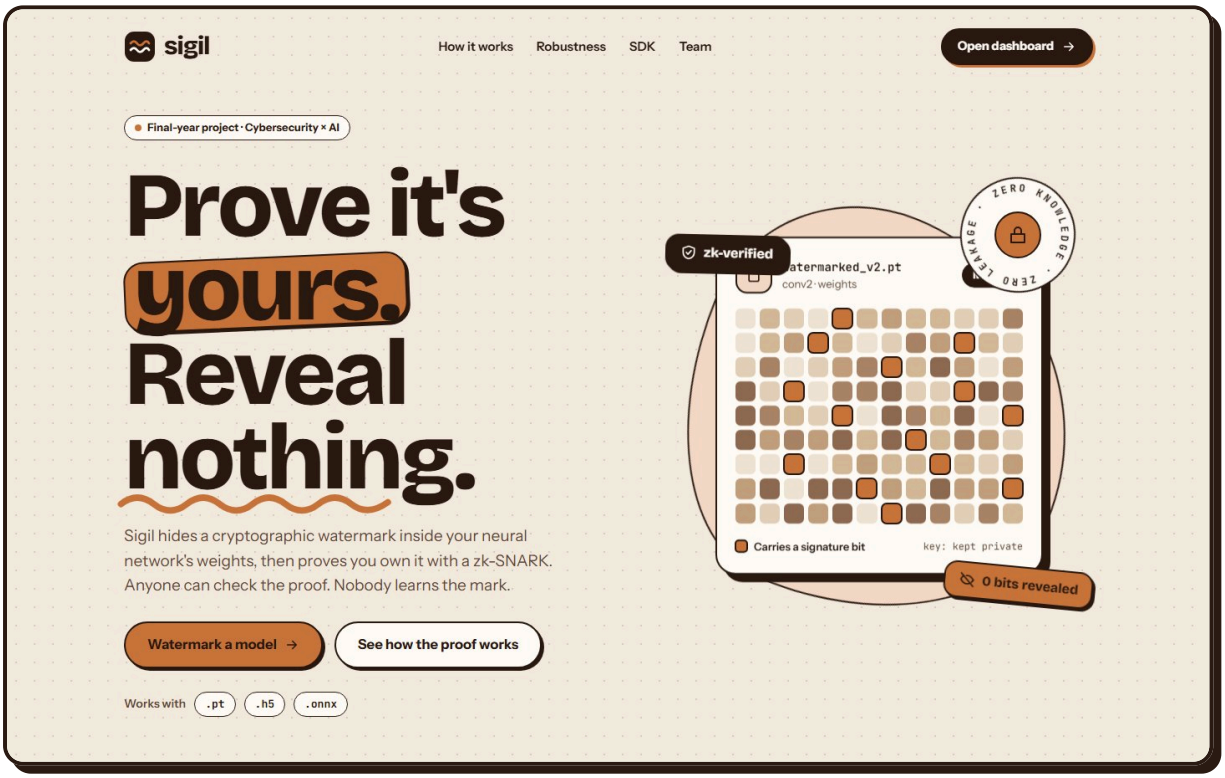
CHAPTER 6

6 A tour of the app

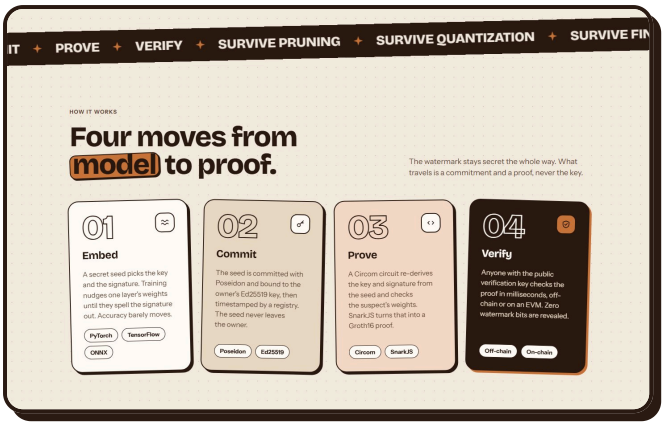
The app has a public-facing home page and a dashboard with four main pages plus the six research demos. Every address works as a bookmark or after a reload.

ADDRESS	PAGE	USE IT TO
`/`	Home	Introduce the project: how it works, robustness, the SDK, the team
`/app`	Overview	See the headline numbers, the models, the proof pipeline and recent activity
`/app/embed`	Embed watermark	Show how the models were watermarked (protocol v1 or v2)
`/app/verify`	Verify ownership	Check the ownership claim, switch off-chain/on-chain, try misuse
`/app/robustness`	Robustness lab	Show what each removal attack does to the model and the mark
`/app/demo/A` … `F`	Research demos	Tell the full research story, one demo per page

6.1 Home page



Home page. **Open dashboard** (top right) goes to the app; **Watermark a model** opens the Embed page; **See how the proof works** scrolls to the four steps. The navigation links jump to How it works, Robustness, SDK and Team.

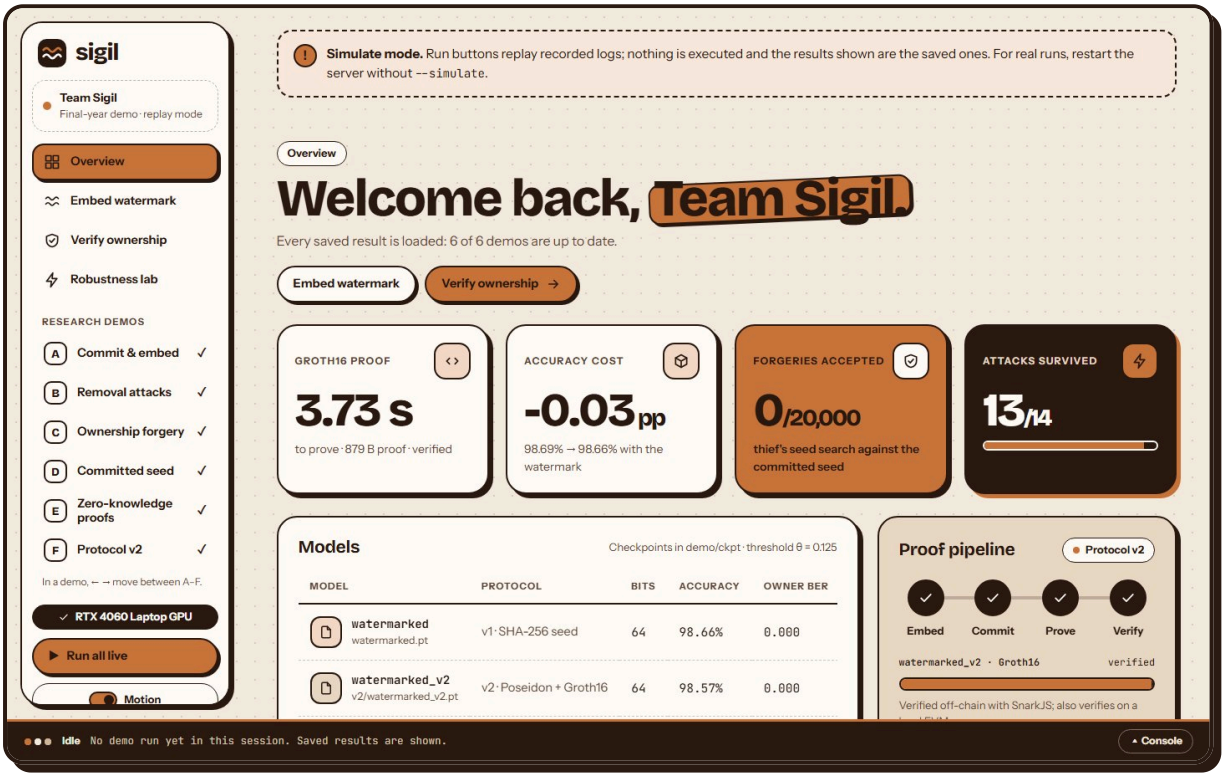


How it works: the four moves from model to proof.



Team: the four members and their roles.

6.2 The app frame: sidebar, banner and console



The dashboard. Sidebar on the left, the page in the middle, the console along the bottom. In replay mode, the dashed banner at the top says so.

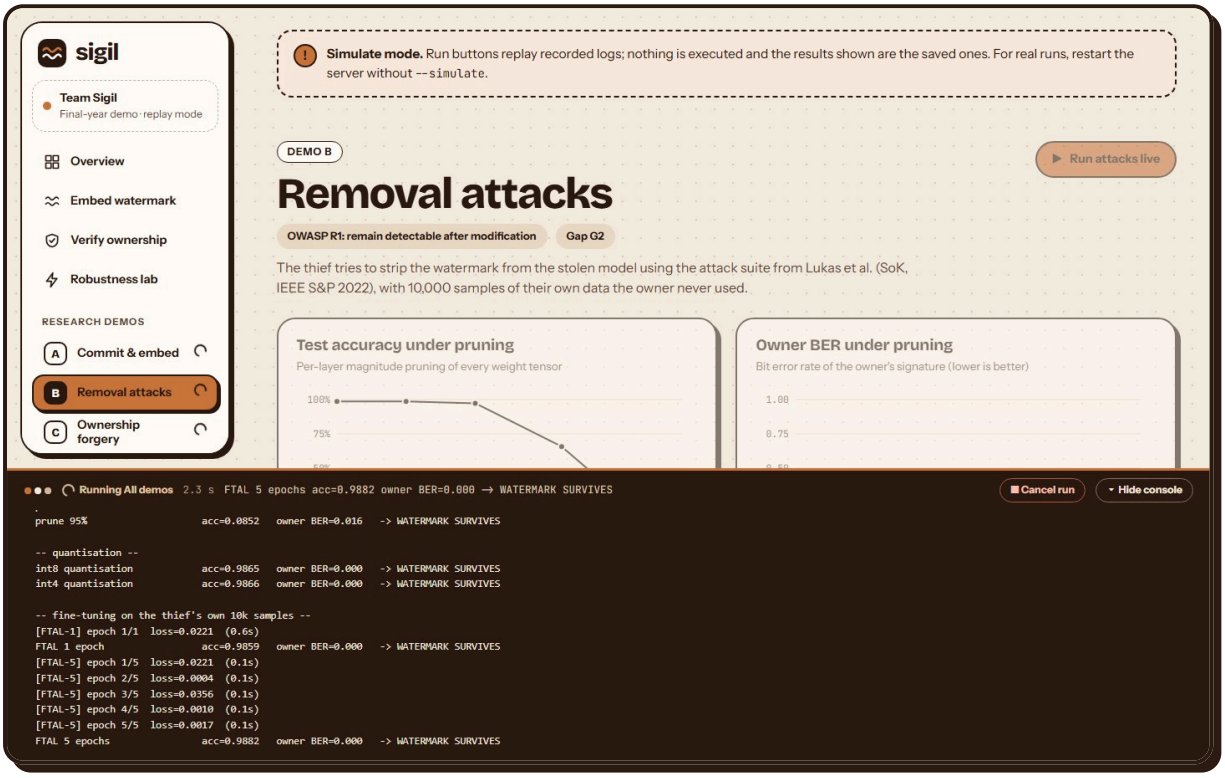
Sidebar

- **sigil** logo: back to the home page. **Team Sigil** card: shows replay or live mode.
- **Pages:** Overview, Embed watermark, Verify ownership, Robustness lab. The current page is highlighted in caramel.
- **Research demos A–F**, each with a status mark:

- ✓ up to date
- ! stale: made with an older owner key (re-run it)
- – freshness unknown
- o no results yet
- ◌ spinning: being refreshed by the current run

- **Environment chip:** your GPU's name, **CPU only**, or **No ML stack: replay only** when PyTorch isn't installed.
- **Run all live:** re-verifies demo A, then runs B, C, D and E1 + E2. Demo F and E3 aren't part of it (run F from its own page; E3 stays precomputed).
- **Motion:** turns the animations off, useful on a slow projector laptop.

Console

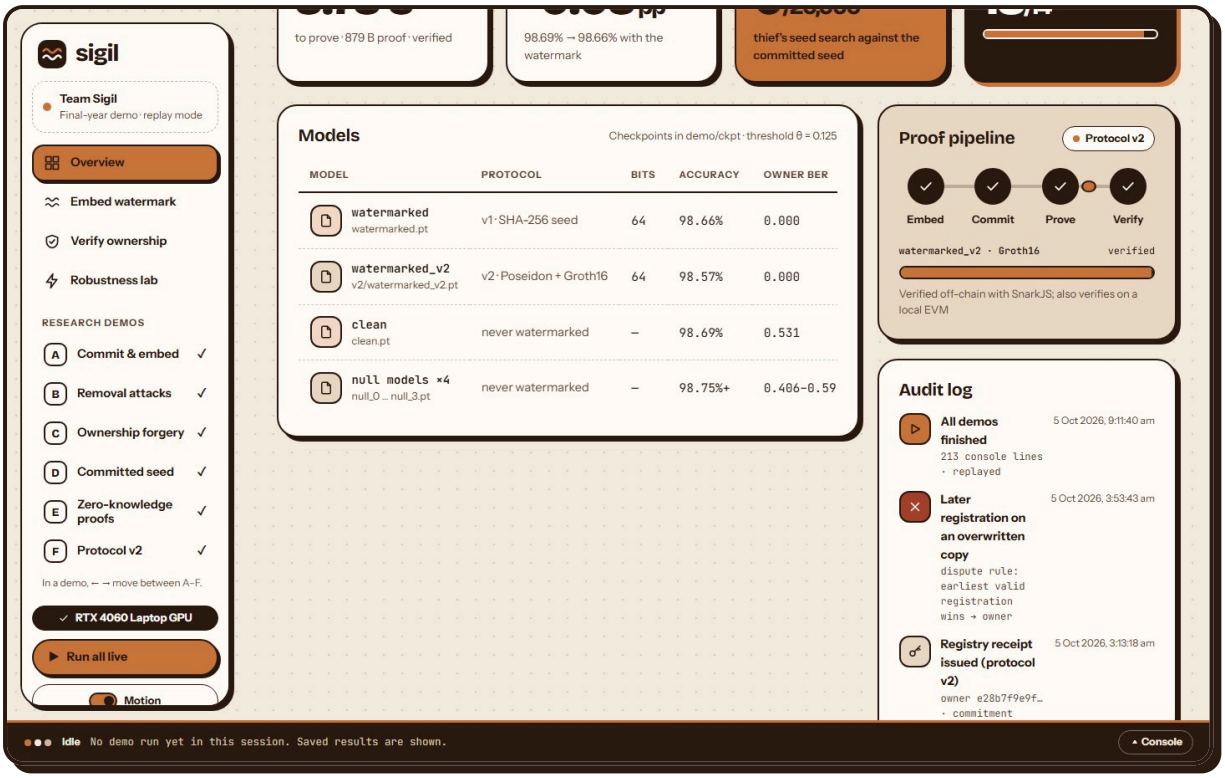


A run in progress. The console opens and streams the script's output. The sidebar shows spinners on the demos being refreshed, and the other Run buttons lock until the run ends. **Cancel run** stops it; **Hide console** collapses it (click the bar to reopen).

CONSOLE STATUS	MEANING
Idle	Nothing has run in this session; the saved results are shown
◌ **Running *name* · seconds**	A run is streaming; the last line is shown next to the timer
✓ ***name* finished**	The run succeeded and the pages reloaded its results
■ ***name* cancelled**	You pressed Cancel run. Results from before the run are kept
✕ ***name* failed (exit *n*)**	The script stopped with an error; its output is in the console

6.3 Overview (`/app`)

The four tiles summarise the project: the **Groth16 proof** time and size, the **accuracy cost** of the watermark, **forgeries accepted** in demo D's seed search, and **attacks survived** in demo B.

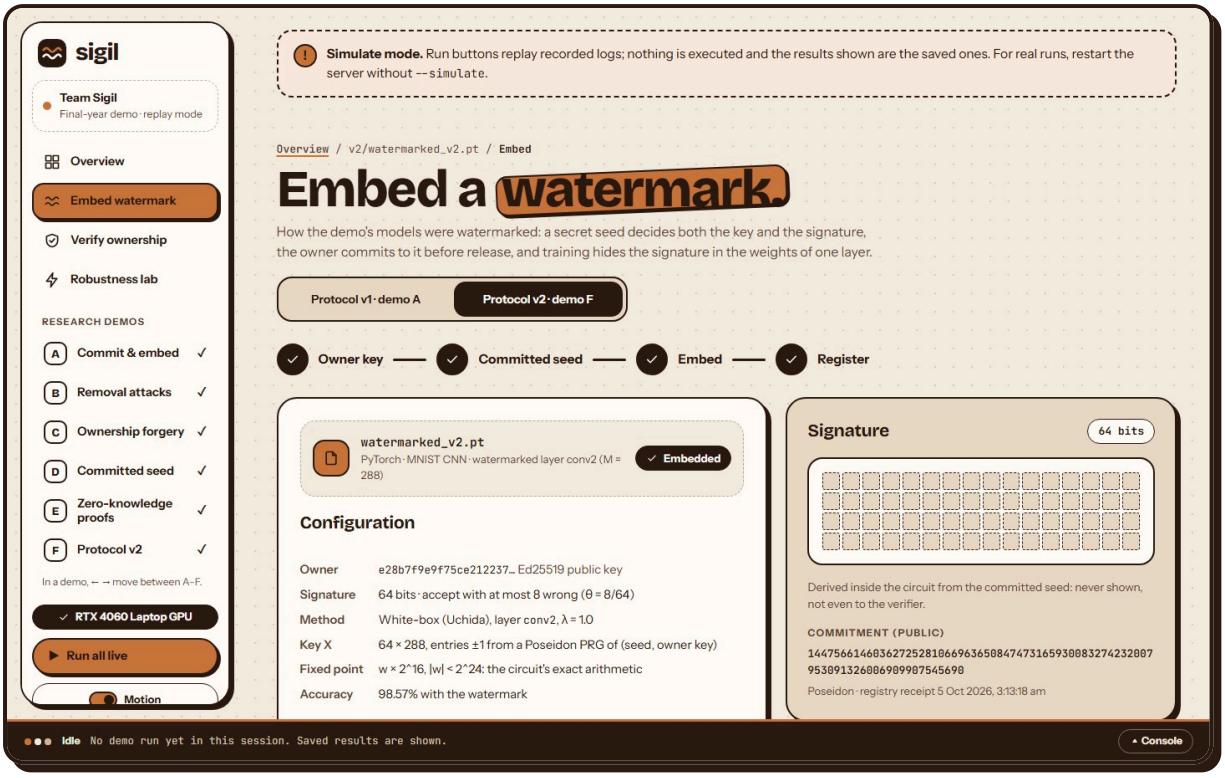


Models, proof pipeline and audit log. The models table lists every checkpoint with its accuracy and owner BER; the last column links to the page that explains it. The pipeline card shows the four protocol steps for the v2 model. The audit log lists recent events from the saved records (commitments, registrations, runs).

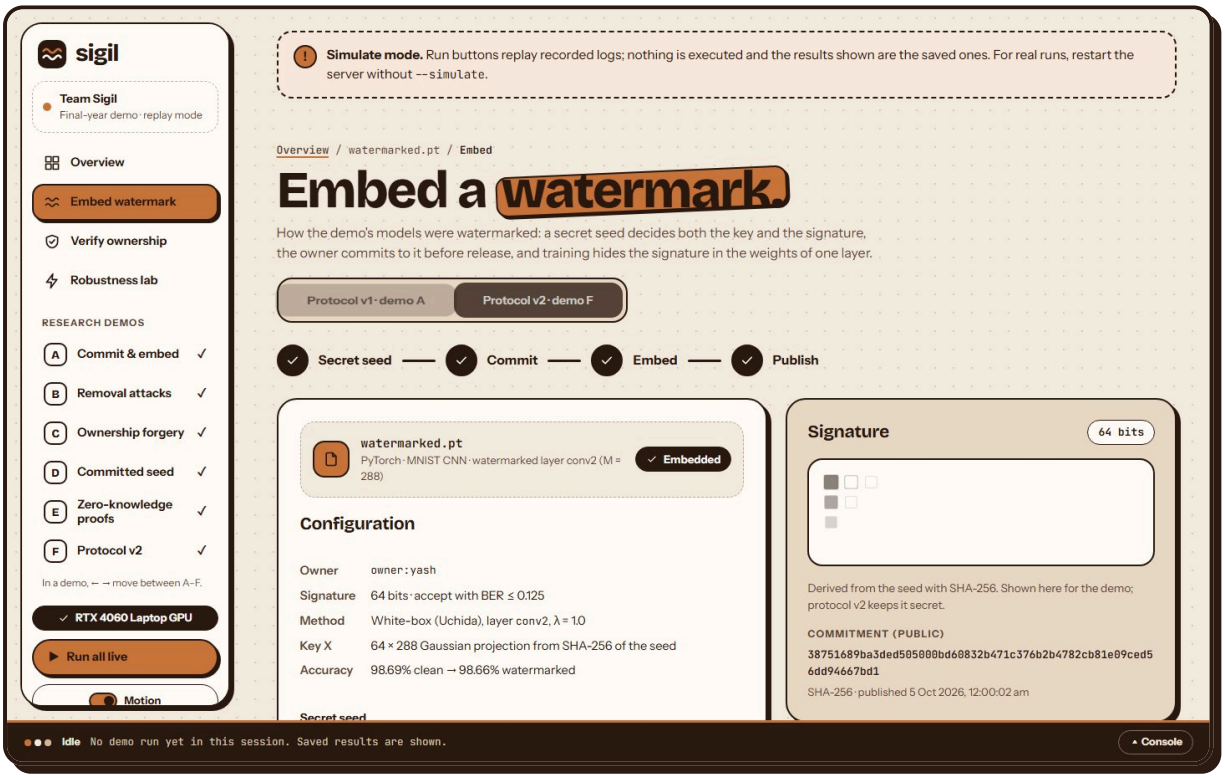
MODEL	WHAT IT IS	ACCURACY	OWNER BER	STATUS
`watermarked.pt`	Protocol v1 watermarked model (demo A)	98.66%	0.000	Ownership shown
`watermarked_v2.pt`	Protocol v2 watermarked model (demo F)	98.57%	0.000	ZK-verified
`clean.pt`	Same network, never watermarked	98.69%	0.531	No match
`null_0-3.pt`	Four independently trained models, never watermarked	≈ 98.8%	0.406–0.594	No match

6.4 Embed watermark (`/app/embed`)

Shows how the demo's models were watermarked. The switch at the top right picks the protocol.

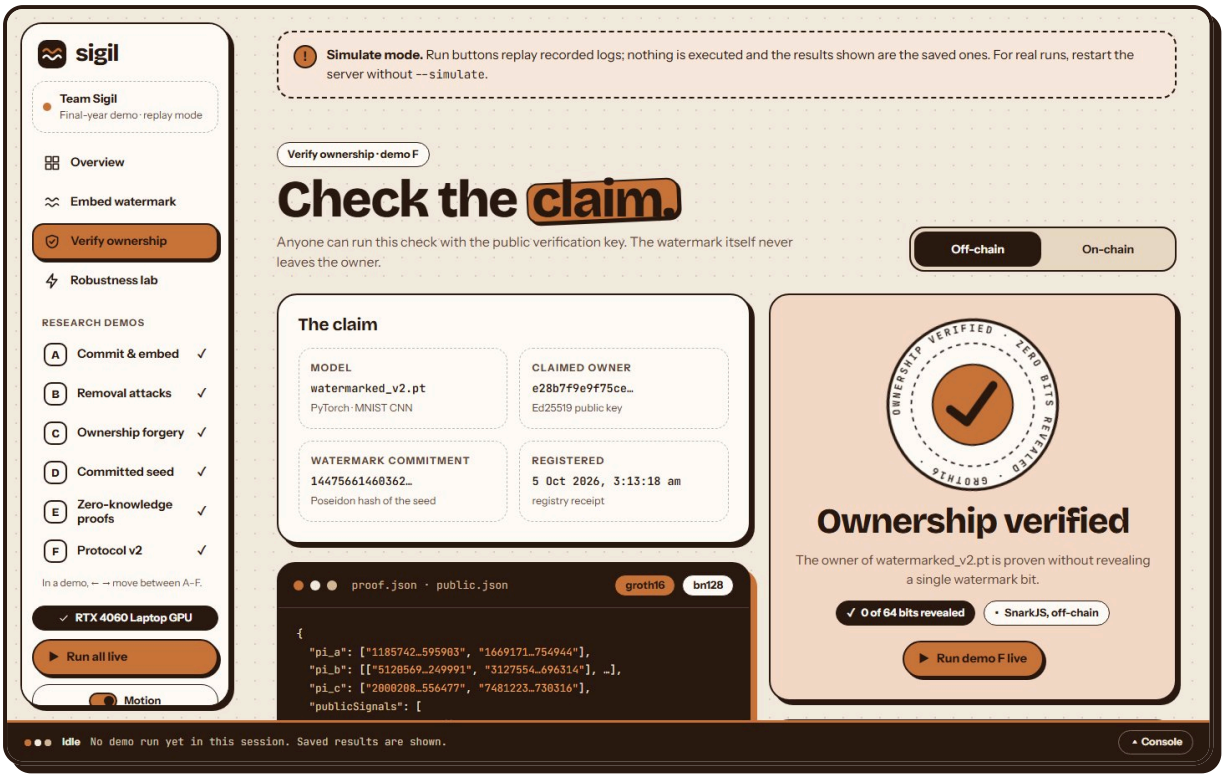


Protocol v2 (demo F). The four steps (Owner key → Committed seed → Embed → Register), the real configuration, the masked secret seed, and the signature drawn as 64 *hidden* squares: in v2 the signature is never shown, not even to the verifier. On the right, the public Poseidon commitment and the registry receipt time. Below, the same flow as `sigil` commands. **Run demo F live** re-runs it; **Verify this model** goes to the Verify page.

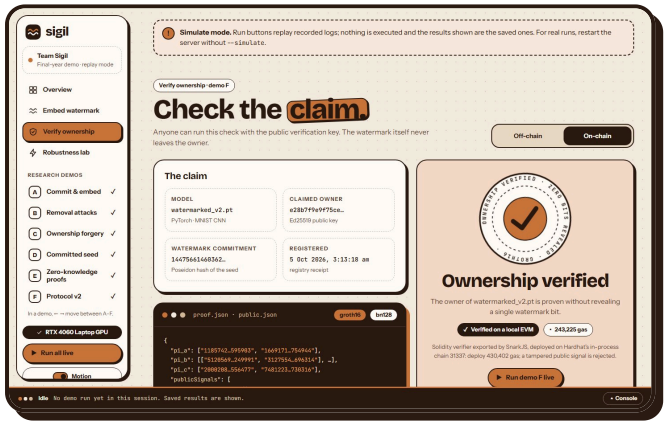


Protocol v1 (demo A). The research baseline. Its 64 signature bits *are* drawn (dark = 1) because v1 is for teaching. **Re-verify live** re-checks the saved models without retraining. **Retrain with a new seed…** asks for confirmation first: it replaces the owner key and makes demos B–E stale until they are re-run.

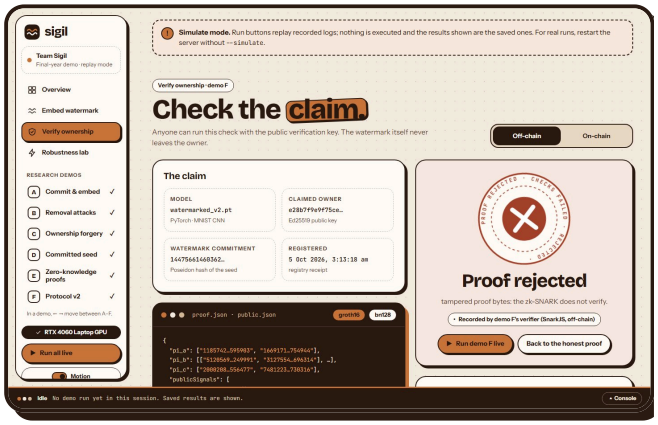
6.5 Verify ownership (`/app/verify`)



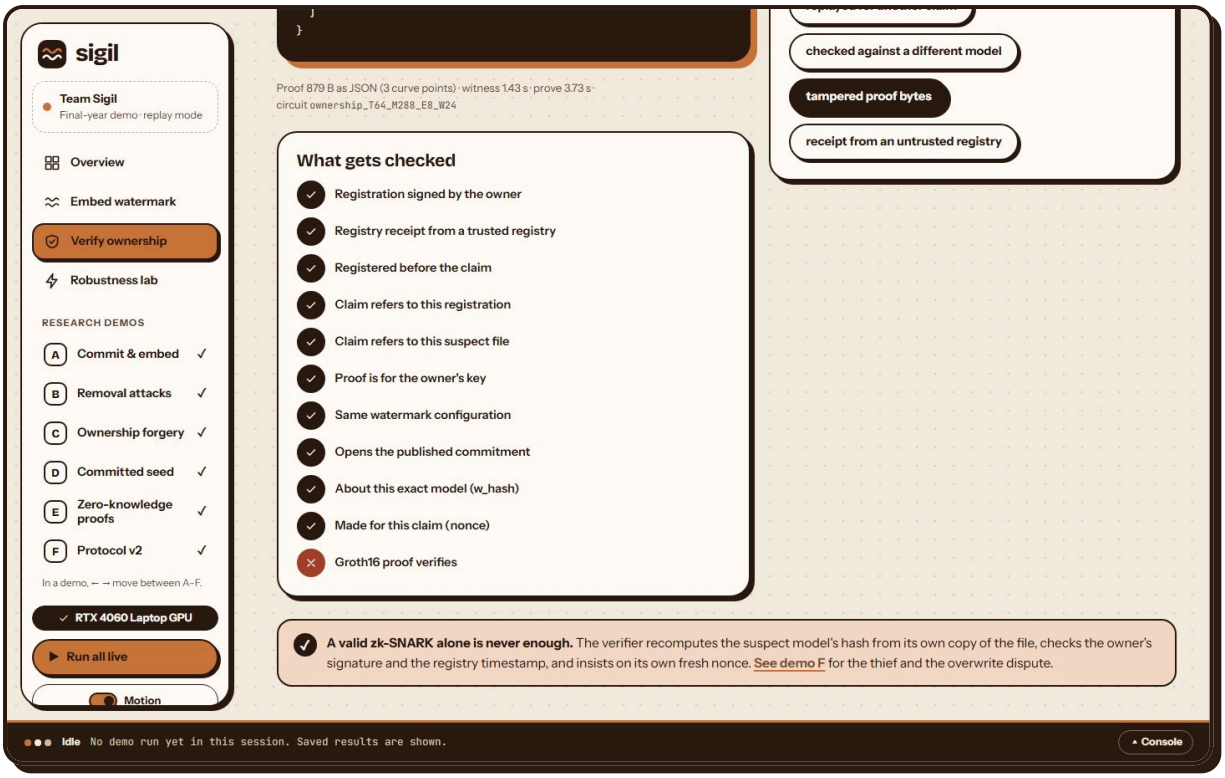
The honest claim. Left: **The claim** (model, claimed owner, commitment, registration time) and the actual `proof.json` / `public.json` from the last demo F run, with the five public signals labelled. Right: the verdict stamp, **Ownership verified**, with "0 of 64 bits revealed".



On-chain. The same proof checked by the Solidity verifier on a local chain: verify gas 243,225, deploy 430,402, chain 31337. A tampered public signal is rejected there too. If the on-chain record belongs to an older proof, the card says so.



A misuse. Clicking **tampered proof bytes** shows what demo F's verifier said about that real attempt: the stamp turns red and shakes, with the reason. **Back to the honest proof** resets it.

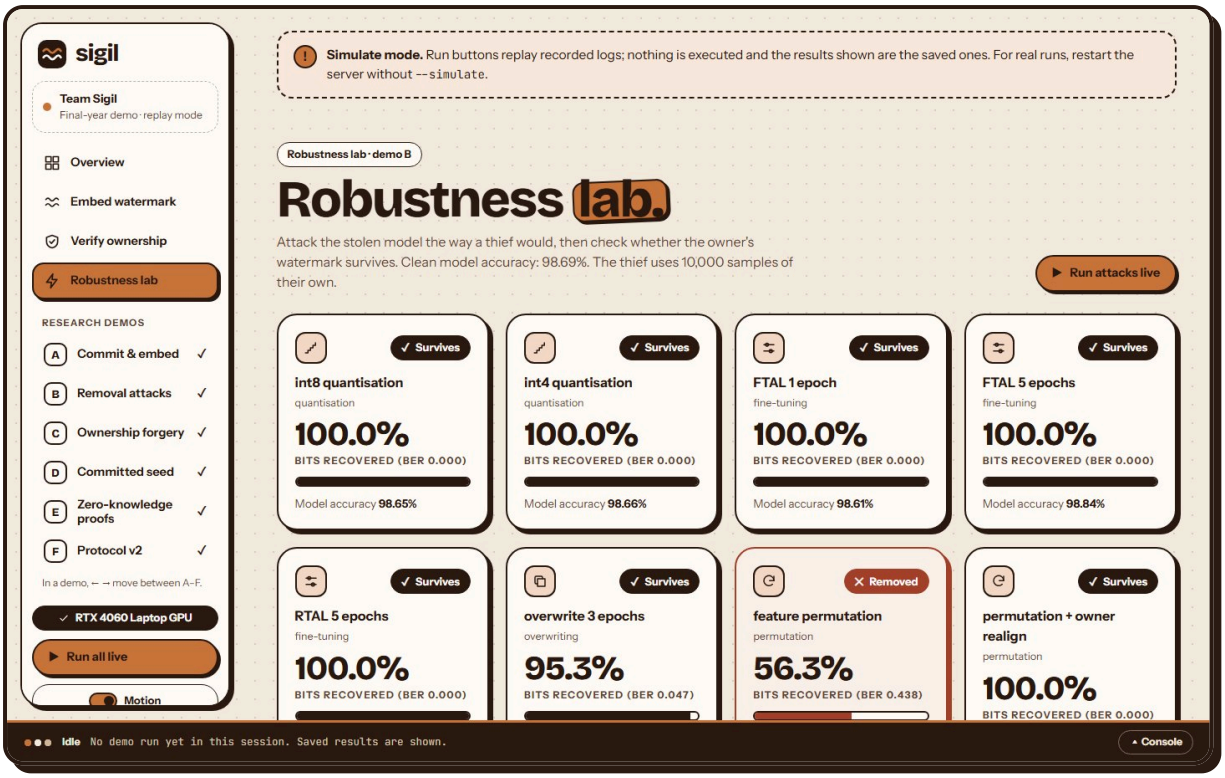


What gets checked. The eleven checks from §3.5. During a misuse, the one that caught it turns red (here: Groth16 proof verifies).

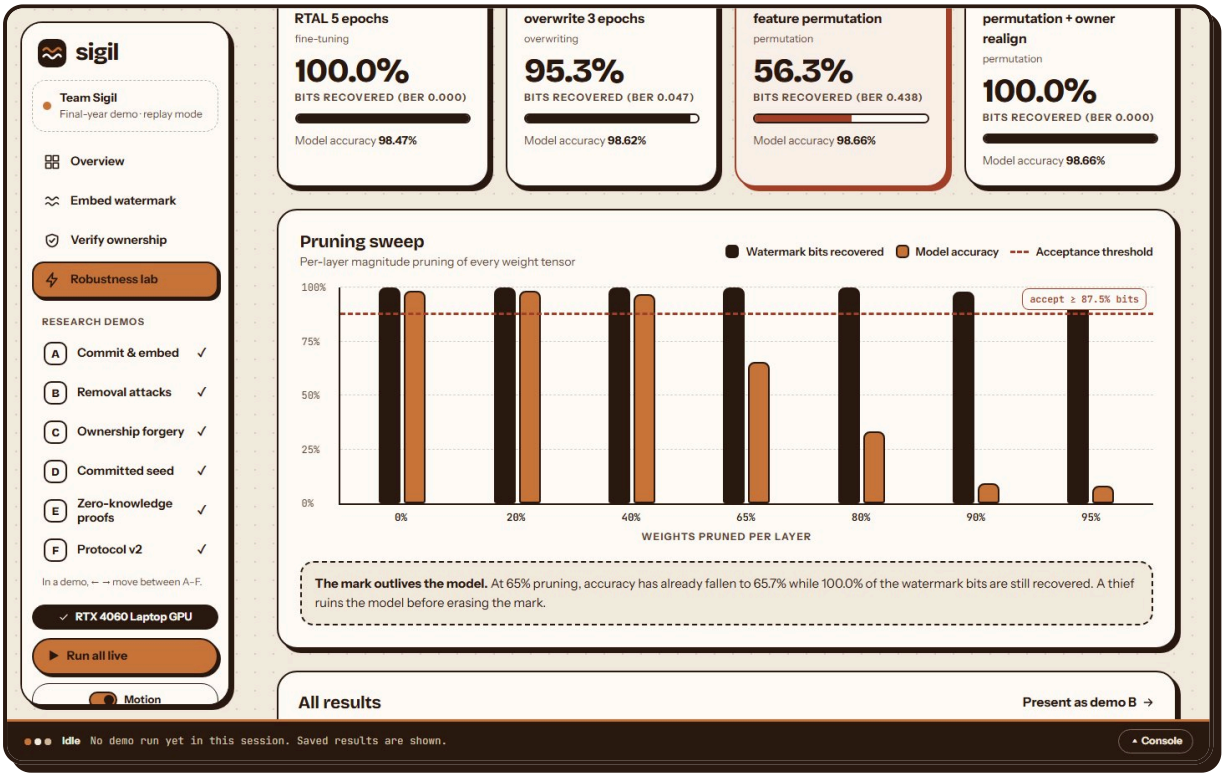
The four misuse buttons

BUTTON	WHAT WAS ATTEMPTED IN DEMO F	CHECK THAT CATCHES IT
replayed for another claim	Re-submit the owner's valid proof for a new claim (different nonce)	Made for this claim (nonce)
checked against a different model	Present the proof against another model file	About this exact model (w_hash)
tampered proof bytes	Change bytes inside the proof	Groth16 proof verifies
receipt from an untrusted registry	Back the registration with a receipt from a registry the verifier doesn't trust	Registry receipt from a trusted registry

6.6 Robustness lab (`/app/robustness`)



Attack cards. Each card is one attack a thief might use on the stolen model: the share of watermark bits still recovered, the BER, and the model's accuracy after the attack. A red card means the attack removed the mark.

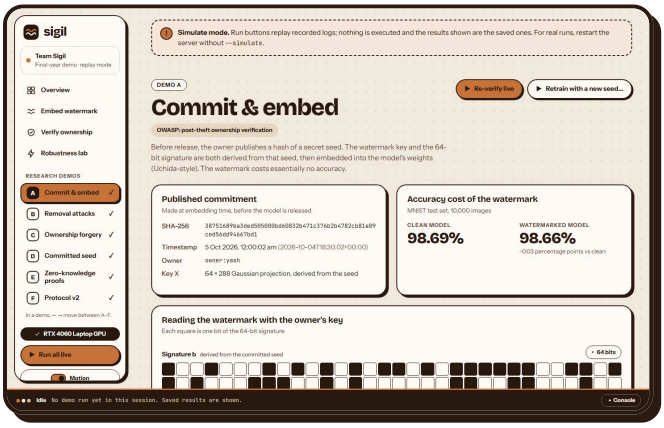


Pruning sweep. Dark bars: watermark bits recovered. Caramel bars: model accuracy. Dashed line: the acceptance threshold (87.5% of bits). The mark outlives the model: at 65% pruning, accuracy has fallen to 65.7% while 100% of the bits are still recovered. Below the chart, **All results** lists every attack, and an **Honest limits** note explains what isn't covered.

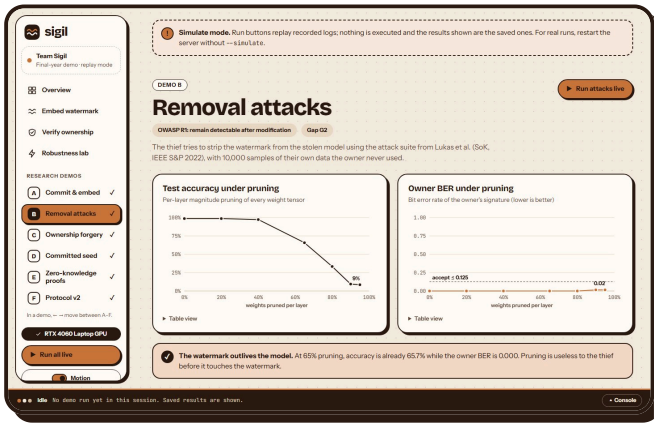
6.7 Research demos A–F (`/app/demo/A` … `F`)

The research storyline, one page per demo, in presentation order. Each page has a title with the requirement it addresses, the panels with real results, a **Run** button, collapsible **Presenter notes** (what to say, including the caveats examiners ask about), and buttons to the previous and next demo. ← → (or a presentation clicker's PageUp / PageDown) move between demos; Home and End jump to A and F.

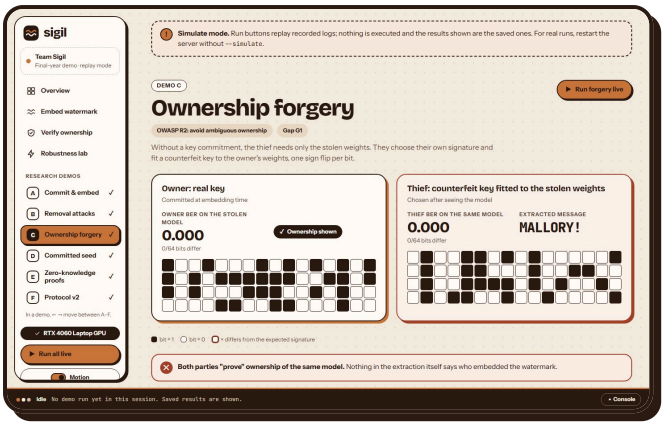
DEMO	QUESTION IT ANSWERS	RESULT
A · Commit & embed	Does hiding a 64-bit signature cost accuracy? Can the owner commit to it before release?	98.69% → 98.66%; owner BER 0.000; clean and null models 0.41–0.59
B · Removal attacks	Can a thief strip the mark (pruning, quantisation, fine-tuning, overwriting, permutation)?	13 of 14 attacks survived; feature permutation removes it unless the owner realigns
C · Ownership forgery	Without a commitment, can a thief claim the model?	Yes: a fitted fake key reads "MALLORY!" with 0 errors, even on never-watermarked models
D · Committed seed	Does committing to the seed stop forgery (without ZK)?	Free signature: thief wins 1,000/1,000. Seed-derived: 0 of 20,000 accepted
E · Zero-knowledge proofs	Is "add a ZK proof" enough on its own?	No: the thief's forged EZKL proof verifies under the owner's key (E2); hashing the parameters fixes it (E3) at a high cost
F · Protocol v2	Does the full design close every gap?	Owner proof accepted; thief can't prove; 4/4 misuse rejected; attacks; dispute won by the earliest registration



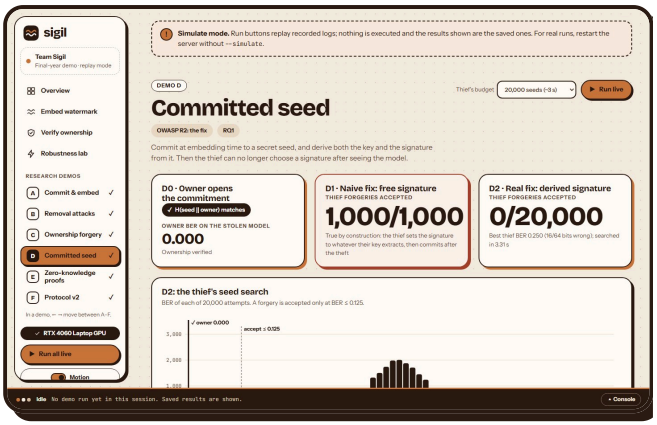
Demo A. The published commitment and the accuracy cost.



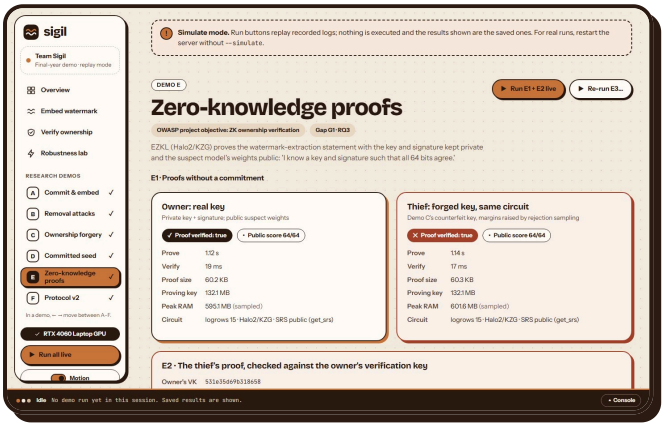
Demo B. Accuracy and owner BER under each attack.



Demo C. The thief's counterfeit key and why it looks real.



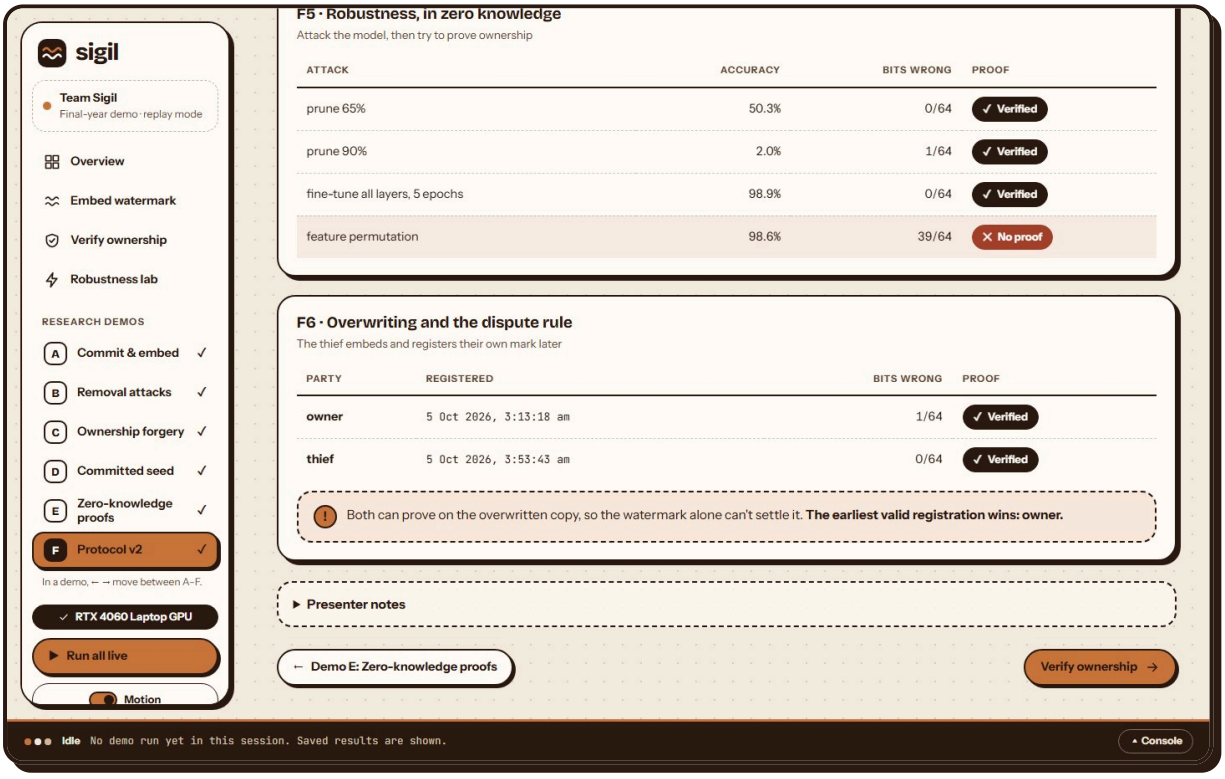
Demo D. Free vs seed-derived signature; the seed search.



Demo E. EZKL proofs; the forged proof that verifies.



Demo F. The full protocol: verdict, timings and every check.



Demo F, end of the page. F5 runs attacks against the v2 proof (feature permutation: "No proof", 39/64 wrong). F6: the thief overwrites with their own mark and registers it later; both can prove, and the dispute rule awards the model to the owner, whose registration is earlier.

CHAPTER 7

7 Step-by-step walkthroughs

W1–W5 work in replay and in live mode; W6 shows an examiner that the runs are real. Times are for presenting.

W1 · The 5-minute presentation

1. **Home** (/), 30 s: read the headline, "Prove it's yours. Reveal nothing." Point at the weight grid: the caramel cells carry signature bits. Click **Open dashboard**.
2. **Overview**, 45 s: walk the four tiles: 3.73 s to prove, −0.03 pp accuracy, 0 of 20,000 forgeries, 13 of 14 attacks survived. In the models table, show that the clean and null models read "No match".
3. **Embed watermark**, 45 s: on protocol v2, point out that the signature is drawn as hidden squares and only the commitment is public. Switch to v1 to show what a revealed signature looks like, then back.
4. **Verify ownership**, 90 s: read the claim, show the proof is tiny (three curve points), then the green stamp. Switch to **On-chain**: 243,225 gas. Click **tampered proof bytes**: red stamp, and the failing check is highlighted. Click **Back to the honest proof**.
5. **Robustness lab**, 60 s: the pruning chart: "the mark outlives the model". Point at the one red card (feature permutation) and say it's the known limit.
6. Close with the research demos if there is time (W5).

W2 · Verify a claim and show a rejection

1. Open **Verify ownership** from the sidebar.
2. Read **The claim** card: which model, whose key, which commitment, when it was registered.
3. In the proof box, note the five public signals: owner key hash, settings hash, commitment, w_hash, nonce. Nothing secret is in them.
4. Check the verdict: **Ownership verified**, 0 of 64 bits revealed. Scroll to **What gets checked**: all eleven are ✓.
5. Under **Try a misuse**, click each button in turn. For each, read the reason on the stamp card and find the red check in the list (see the table in §6.5).
6. Toggle **Off-chain / On-chain** to show the same proof checked by the smart contract.

W3 · Run a demo and cancel a run

1. Open a demo page, for example **Demo F**, and click its caramel **Run** button (or **Run all live** in the sidebar).
2. The console opens and streams the output. In live mode demo F takes about 1–2 minutes (86 s on a CPU-only laptop); in replay mode about 2 s.
3. While it runs, the sidebar shows spinners on the affected demos and the other Run buttons are locked.
4. To stop, click **Cancel run** in the console. The status turns to "cancelled" and the previous results stay.
5. When it finishes, the status shows ✓ and every page reloads the new results.

Don't run these live in front of an audience

Retrain (demo A) creates a new owner seed and key, which makes demos B–E stale until re-run. **E3** needs about 32 GB of RAM and 10 GB of disk. Both ask for confirmation first; use replay mode or the saved results for them.

W4 · Show what attacks do

1. Open **Robustness lab**.
2. Pick a card, for example **int4 quantisation**: 100% of bits recovered, accuracy 98.66%. The thief gains nothing.
3. Scroll to the **Pruning sweep**: as pruning grows, accuracy (caramel) collapses long before the bits (dark) drop below the dashed threshold.
4. Show the red **feature permutation** card: accuracy unchanged, 56.3% of bits (BER 0.438): removed. Then **permutation + owner realign**: back to 100%.
5. Click **Run attacks live** to re-run demo B (about 10 s live).

W5 · Present the research story A → F

1. Open **Research demos** → **A** in the sidebar. Use → (or a clicker) to advance.
2. **A:** the owner commits, then embeds; accuracy is unchanged.
3. **B:** the mark survives the usual attacks.
4. **C:** but without a commitment, a thief forges ownership of anything ("MALLORY!").
5. **D:** committing to the seed stops that: 0 of 20,000.

6. **E:** adding a ZK proof naively isn't enough: the forged proof verifies too.
7. **F:** protocol v2 binds everything (owner key, settings, commitment, model, claim) and closes every gap shown before. Open **Presenter notes** on each page for the exact wording and caveats.

W6 · Show the teacher it's really live

Replay mode only *shows* saved results. In live mode every Run button really trains, attacks and proves on the laptop in front of the examiner, and the numbers on screen change. This is how to make that visible.

The day before

1. On the presentation laptop, run the setup once (§5.3, Option 2). It needs the internet; the presentation itself doesn't.
2. Start with `Start-Demo-UI.cmd`, *not* the Simulate one. Check three things: there is **no "Simulate mode" banner**, the sidebar card says **live mode**, and the environment chip shows the GPU or **CPU only**.
3. Do one rehearsal run of demo F, so the first run in front of the teacher isn't the first run ever.

In front of the teacher

1. **Note the "before".** On Overview, read out "0 / 20,000" forgeries. On Verify ownership, read out the first digits of `pi_a` in the proof box and the registration time.
2. **Let the teacher choose.** Open **Demo D**, let the teacher pick the number of thief seeds (20,000 or 100,000) and click **Run live**. The console streams the real search (about 3 s for 20,000, about 17 s for 100,000). Afterwards the Overview tile reads, for example, "0 / 100,000". Measured on this laptop: 100,000 seeds in 16.8 s, 0 accepted.
3. **Generate a fresh proof.** Open **Demo F** and click **Run demo F live** (about 80 s on a GPU laptop, 86 s CPU-only). Explain the six parts while the console streams them. When it finishes, the Verify page shows a *brand-new* proof: different `pi_a` digits, a new nonce, a new registration time, and new entries in the Overview audit log.
4. **Attack it live.** On **Robustness lab**, click **Run attacks live** (about 10 s): pruning, quantisation and fine-tuning run again and the chart redraws.
5. **Optional, strongest:** on **Embed watermark** → **Protocol v1**, click **Retrain with a new seed…** (about 35 s on a GPU). A new owner seed, a new commitment and new signature bits appear. Demos B–E are then marked **!** (stale) until you click **Run all live** (about 30 s), which recomputes them with the new key. Do this last.

What stays precomputed, and why

E3 needs about 32 GB of RAM, so it's shown from the saved run. The **on-chain** check runs separately from `circuits/onchain`: after a live demo F run, the On-chain view says it was made for an earlier proof. That's expected; the off-chain verdict is the live one. Live runs overwrite `demo/results/`; to get the shipped results back, unzip the package again.

CHAPTER 8

8 The SDK and command line

The `sigil` SDK is what the app demonstrates, packaged for real use. It is installed by `setup.ps1 --demo` / `setup.sh --demo`. Run the commands from the activated environment (`.venv\Scripts\activate` on Windows, `source .venv/bin/activate` elsewhere), or as `python -m sigil.cli …`.

8.1 Commands

ROLE	COMMAND	WHAT IT DOES
Owner	`sigil keygen`	Create the owner's Ed25519 key, encrypted (`secrets/owner.key`)
Owner	`sigil seed --bits 64 --layer conv2`	Create the secret seed for a configuration, encrypted (`secrets/seed.json`); prints the commitment. Options: `--m` (layer vector length, default 288), `--max-err` (default T/8)
Owner	`sigil register model.pt`	Sign a registration for the watermarked file (`registration.json`). With `--registry-key`, also timestamp it locally
Owner	`sigil detect model.pt`	Check the watermark in a file: bits wrong, BER, accepted
Owner	`sigil prove suspect.pt --claim claim.json`	Make the zero-knowledge proof for a claim (writes a `bundle/` folder)
Verifier	`sigil claim registration.json --suspect suspect.pt`	Open a claim with a fresh nonce (`claim.json`)
Verifier	`sigil verify bundle --suspect … --registration … --receipt … --trusted-registry <hex>`	Run every check of §3.5 and print the verdict with reasons
Registry	`sigil registry-keygen`, `sigil receipt registration.json`	Create a registry key; timestamp a registration
Toolchain	`sigil zk-setup`, `sigil zk-info`	Build the circuit's proving keys (about 3 min); print the circuit's size

Add `--json` before the command for machine-readable output. Passphrases are asked for at a prompt, or read from `SIGIL_PASSPHRASE` (owner) and `SIGIL_REGISTRY_PASSPHRASE` (registry).

8.2 An end-to-end session

```
# --- owner, before release ---
sigil keygen
sigil registry-keygen                          # a local registry, for testing
sigil seed --bits 64 --layer conv2
# train with sigil.embed.embed_torch(...) (see 8.3), save model.pt
sigil detect model.pt                          # expect: accepted, 0 bits wrong
sigil register model.pt --registry-key secrets/registry.key
#   → registration.json and registration.receipt.json

# --- later: a verifier checks a suspect copy ---
sigil claim registration.json --suspect suspect.pt             # verifier
sigil prove suspect.pt --claim claim.json                      # owner → bundle/
sigil verify bundle --suspect suspect.pt --registration registration.json \
      --receipt registration.receipt.json --trusted-registry <registry public key hex>
```

8.3 From Python

```
from sigil import protocol as P
from sigil.identity import KeyPair
from sigil.embed import embed_torch

owner = KeyPair.generate()
cfg   = P.Config(T=64, layer="conv2")          # M=288, max_err=8 by default
seed  = P.new_seed()
pkh   = P.pk_hash(owner.public_bytes)
commitment = P.commitment(seed, pkh, cfg)        # publish this before release

# train a torch.nn.Module with the watermark on cfg.layer
model, detection = embed_torch(model, (x_train, y_train), cfg, seed, pkh, epochs=2)
print(detection.errors, detection.accepted)
```

8.4 Model formats and safety

- **Formats:** PyTorch `.pt`, Keras `.h5` / `.keras` and `.onnx`. The same weights give the identical layer vector in every format.
- **Safe loading:** PyTorch files load with `weights_only=True`, Keras with `safe_mode=True` (Lambda layers refused), ONNX through `onnx.checker` with size limits. `--trust` turns this off: **only for your own files, never for a suspect's**.
- **ONNX caveat:** exporters fold BatchNorm into the convolution before it, changing that layer. Verify from the original `.pt` / `.keras` file, or export without constant folding.
- **Secrets:** keep `secrets/` off shared drives and out of chats. The key and signature are derived in memory and never written to disk.

CHAPTER 9

9 Results at a glance

MNIST classifier, watermark in `conv2` (288 values), T = 64 bits, accept at ≤ 8 wrong (θ = 0.125). Saved results from the last full run on the team's laptop (RTX 4060).

Demo A · Accuracy and detection

MODEL	ACCURACY	OWNER BER	VERDICT
Clean (never watermarked)	98.69%	0.531	**No match**
Watermarked (v1)	98.66%	0.000	**Ownership shown**
Null models 0–3	98.75–98.80%	0.484 · 0.594 · 0.406 · 0.453	**No match**

Demo B · Removal attacks (thief uses 10,000 samples of their own)

ATTACK	MODEL ACCURACY	OWNER BER	WATERMARK
None (stolen as-is)	98.66%	0.000	**Survives**
Prune 20% / 40%	98.60% / 97.28%	0.000 / 0.000	**Survives**
Prune 65% / 80%	65.73% / 33.35%	0.000 / 0.000	**Survives**
Prune 90% / 95%	9.41% / 8.52%	0.016 / 0.016	**Survives**
int8 / int4 quantisation	98.65% / 98.66%	0.000 / 0.000	**Survives**
Fine-tune all layers (FTAL) 1 / 5 epochs	98.61% / 98.84%	0.000 / 0.000	**Survives**
Reset the last layer, re-train all (RTAL) 5 epochs	98.47%	0.000	**Survives**
Overwrite with own mark, 3 epochs	98.62%	0.047	**Survives**
Feature permutation	98.66%	0.438	**Removed**
Permutation + owner realign	98.66%	0.000	**Survives**

Demos C and D · Forgery

C: thief's fitted key on the stolen model	Reads "MALLORY!", BER 0.000. Also 0.000 on the clean and all four null models
C: does the fake key look real?	Yes: entry distribution KS distance 0.0104, below the 0.0142 critical value; a large-margin variant matches the owner's margins (1.43 vs 1.42)
D0: owner with committed seed	Commitment matches, BER 0.000: verified
D1: commitment, but free signature	Thief wins 1,000 of 1,000 (the signature isn't tied to the seed)
D2: signature derived from the seed	0 of 20,000 thief seeds accepted (best BER 0.250); 2.8×10^{-10} per try; $\approx 2^{32}$ work, about 114 single-core hours for a 50% chance at T = 64; $\approx 2^{120}$ at T = 256

Demo E · EZKL zero-knowledge experiments

VARIANT	PROVE	VERIFY	PROOF	PROVING KEY	NOTE
E1 private parameters	1.1 s	19 ms	≈ 60 KB	138 MB	Proof of extraction
E2 thief, same setup	1.1 s	17 ms	≈ 60 KB	138 MB	**Forged proof verifies under the owner's key**
E3 Poseidon-hashed parameters	39–47 s	0.56–0.58 s	≈ 66 KB	10.5 GB	Thief's hash doesn't match; precomputed (≈ 32 GB RAM)

Demo F · Protocol v2 (Circom + Groth16)

Circuit	`ownership_T64_M288_E8_W24`, 116,822 constraints
Watermarked v2 model	Accuracy 98.57%; owner detection 0 of 64 wrong
Owner proof	Witness 1.43 s, prove 3.73 s, 879-byte proof; all checks pass: **accepted**
On-chain (local Hardhat chain 31337)	Verify 243,225 gas; deploy 430,402 gas; a tampered public signal is rejected
Thief	Own key: 28 of 64 wrong, no proof possible. Owner's commitment: can't be opened
Misuse	4 of 4 rejected (replay, different model, tampered proof, untrusted registry)
F5 attacks with proof	Prune 65%: accuracy 50.26%, 0 wrong, accepted. Prune 90%: 2.01%, 1 wrong, accepted. Fine-tune all layers 5 epochs: 98.88%, 0 wrong, accepted. Feature permutation: 98.57%, 39 wrong, **no proof**
F6 overwrite dispute	Owner (registered first) 1 of 64 wrong, proof accepted; thief (registered later) 0 of 64 wrong, proof accepted: **winner: owner**, by the earliest registration

CHAPTER 10

10 Pass and fail cases

The demo deliberately shows failures as well as successes. All of them are real recorded runs.

Accepted (should pass, and does)

- The owner's proof for the watermarked v2 model: all eleven checks pass, off-chain and on-chain.
- The owner's mark after pruning (up to 95%), int8/int4 quantisation, fine-tuning and overwriting.
- The owner's mark after feature permutation, *once the owner realigns the channels*.

Rejected (should fail, and does)

CASE	WHERE TO SEE IT	WHY IT FAILS
Replayed proof	Verify → Try a misuse	Nonce mismatch
Proof checked against another model	Verify → Try a misuse	w_hash mismatch
Tampered proof bytes	Verify → Try a misuse	The zk-SNARK doesn't verify
Receipt from an untrusted registry	Verify → Try a misuse	Registry receipt invalid
Thief proving with their own key	Demo F (F3)	28 of 64 bits wrong (> 8)
Thief opening the owner's commitment	Demo F (F3)	They don't have the seed
Thief's seed search	Overview tile, demo D	0 of 20,000 accepted
Never-watermarked models	Overview models table, demo A	BER 0.41–0.59, far above 0.125
Tampered public signal on-chain	Verify → On-chain	The Solidity verifier rejects it

Shown failing on purpose: weaknesses of naive schemes

- **Demo C:** without a commitment, the thief's forgery succeeds, even on models that were never watermarked.
- **Demo D1:** committing to a seed but keeping the signature free: the thief wins every time.
- **Demo E2:** a zero-knowledge proof that isn't bound to the owner's parameters: the thief's forged proof verifies.
- **Feature permutation** (demo B, F5): removes the white-box mark unless the owner realigns first.
- **Overwriting** (demo F6): both parties produce valid proofs; only the registry timestamp settles it.

CHAPTER 11

11 Honest limits

What the demo does not claim. These are also in the app's Presenter notes, because examiners ask about them.

- **White-box only.** The verifier needs the suspect's weights. A copy made only by querying the model's outputs (model extraction) isn't covered by a white-box mark (OWASP limitation L1).
- **Feature permutation** reorders channels at no accuracy cost and removes the mark unless the owner realigns before proving.
- **T = 64 is a demo size.** Forging by seed search costs about 2^{32} tries (about 114 single-core hours for a 50% chance): feasible with GPUs. Timestamps stay essential; T = 256 raises it to about 2^{120}.
- **The threshold is indicative.** 8 of 64 is not a calibrated false-positive rate; four null models can't give one.
- **Overwriting** produces two valid marks. The registry timestamp, not the watermark, decides who was first.
- **Trusted setup.** Phase 1 uses the public Perpetual Powers of Tau; its full transcript check was too slow for the demo laptop and is recorded as pending in the setup files. Phase 2 has a single contributor. Groth16 is sound if at least one contributor in each phase discarded their randomness.
- **Research demos C and E.** Demo C runs ZKROWNN's *relation*, not ZKROWNN's code (ZKROWNN assumes a semi-honest prover). E3 is the halfway fix and needs about 32 GB of RAM, so it is precomputed.
- **Demo identities are public.** Seeds and keys in `demo/ckpt/` are committed for reproducibility: anyone with the repository can produce the demo owner's proofs.

CHAPTER 12

12 Troubleshooting and FAQ

PROBLEM	FIX
Double-clicking the launcher shows "python is not recognized"	Install Python 3.10+ from python.org and tick "Add python.exe to PATH", then try again.
The browser didn't open	Open `http://127.0.0.1:8765` yourself. The console window must stay open.
"Can't reach the demo server"	The server isn't running, or port 8765 is taken: start it again, or use `python demo/server.py --port 8770`.
PowerShell refuses to run `setup.ps1`	`powershell -ExecutionPolicy Bypass -File setup.ps1 --demo`
Sidebar says "No ML stack: replay only"	PyTorch isn't installed for the Python the server uses. Run the setup (Option 2), start with `Start-Demo-UI.cmd`, or present in replay mode.
Demo F says the circuit isn't set up	Re-run the setup (it installs `dist/sigil-proving-keys-t64.zip`), or run `.venv\Scripts\python -m sigil.cli zk-setup` (about 3 minutes).
Antivirus blocks `tools/bin/circom.exe`	It's the official circom 2.2.3 release from github.com/iden3/circom. Allow it, or present demo F in replay mode.
A demo shows "!" (stale) in the sidebar	It was made with an older owner key (after a Retrain). Re-run it, or **Run all live**.
The page looks old or the fonts look plain	Hard-refresh with Ctrl+F5. The fonts are bundled; nothing is loaded from the internet.
Animations stutter on the projector laptop	Switch **Motion** off in the sidebar.

Frequently asked

Does it need the internet, or send data anywhere?

No. The app and the server run on your machine and listen only on `127.0.0.1`. Only the optional setup downloads packages.

Is replay mode faking the results?

No. The pages show the saved results of real runs, and the console replays their recorded output. The banner says it is replay mode. Note that the recorded console logs and the saved result files come from runs made at slightly different times, so a few printed values can differ by a bit or two (for example, demo F6's owner shows 0 of 64 wrong in the replayed log and 1 of 64 in the saved results). Live mode always shows the current run.

Can I watermark my own model?

Yes, with the SDK (chapter 8): PyTorch training via `embed_torch`, Keras via the Keras adapter, and detection and verification from `.pt`, `.h5` / `.keras` or `.onnx` files.

What exactly does the verifier learn?

Five public values: hashes of the owner key and settings, the commitment, the hash of the suspect model's layer, and the nonce the verifier chose itself. Not the seed, the key or a single signature bit.

Why is it called Sigil?

A sigil is a seal: a mark that shows something belongs to someone. Here the seal is hidden in the weights and proven without being shown.

CHAPTER 13

13 Team and credits

MEMBER	ROLL NO.	ROLE	BUILT
Sami Daire	231813	Project lead · ZK & verification	Circom circuits, SnarkJS verification, the protocol and its integration
Mohamed Ansari	231803	AI & watermarking	Embedding and detection in PyTorch and TensorFlow; the attacks of the Robustness lab
Arfat Shaikh	231846	Frontend & integration	The React dashboard, wiring it to the backend APIs, packaging the Python SDK
Yash Bandiwadekar	231810	Backend developer	The backend services, the demo server and the APIs for watermark tracking and verification history

Guided by Er. Fiza Shaikh (Co-Guide) and the Department Review Committee.

Built on

- **Uchida et al.**, "Embedding Watermarks into Deep Neural Networks" (white-box watermark).
- **Lukas et al.**, "SoK: How Robust is Image Classification Deep Neural Network Watermarking?" (IEEE S&P 2022): the attack suite of demo B.
- **Circom** and **SnarkJS** (iden3), **circomlib**'s Poseidon; the **Perpetual Powers of Tau** (Privacy & Scaling Explorations).
- **EZKL** for the demo E experiments; **Hardhat** for the local chain; **PyTorch**, **TensorFlow/Keras** and **ONNX**.
- The **OWASP AI Model Watermarking** incubator project, whose goals this student project follows. Sigil is not official OWASP work.

In one sentence

Sigil lets a model's owner prove "this model is mine" to anyone, with a 879-byte proof that reveals nothing about the watermark.